SOA : SERVICES WEB SOAP
TP N°1

AUDITOIRE : 3 EME ANNEE LI

Les services Web fonctionnent sur un modèle client-serveur où ils communiquent via le réseau. Le composant côté serveur fournit l'URL du terminal client où se trouve le service et où l'application cliente peut appeler différentes méthodes.

Il existe deux types de services Web:

1. Services Web SOAP
2. Services Web Rest

Nous utiliserons JAX-WS pour créer des services Web basés sur SOAP. JAX-WS est synonyme d'API Java pour les services Web XML. JAX-WS est une API Java basée sur XML permettant de créer un **serveur** de services Web et une application client. Cela fait partie de l'API Java standard, nous n'avons donc pas besoin d'inclure quoi que ce soit qui fonctionne avec elle.

Maintenant que nous avons examiné la terminologie des services Web.

Nous allons créer un service Web qui exposera des méthodes pour ajouter, supprimer et récupérer des objets **Person** (addPerson, deletePerson, getPersonByName, getPerson, getAllPersons). Cette personne est caractérisée par : un **name**, **age** et **identifiant**.

Tout d'abord, nous allons créer un **modèle** pour nos données dans un package nommé : **com.info.model.** Cette classe est nommée :

`Person.java`

1) Nous allons maintenant devoir créer une **interface** dans un nouveau package nommé : **com.info.service**, où nous allons déclarer les méthodes que nous exposerons dans nos exemples de services Web JAX-WS. Cette interface nommée :

`PersonService.java`

```java
import javax.jws.WebMethod;

@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface PersonService {

    @WebMethod
    public boolean addPerson(Person p);

    @WebMethod
    public boolean deletePerson(int id);

    @WebMethod
    public Person getPersonByName(String name);

    @WebMethod
    public Person getPerson(int id);

    @WebMethod
    public Person[] getAllPersons();
}
```

Notez que vous allez utiliser les annotations `@WebService` et `@SOAPBinding` de l'API JAX-WS. Nous pouvons créer des services Web SOAP en **style RPC** ou en **style Document**. Nous pouvons utiliser n'importe lequel pour créer des services Web. La différence réside dans la manière dont le **fichier WSDL est généré**.

2) Nous allons maintenant écrire la **classe d'implémentation** nommée :

`PersonServiceImpl.java`

```java
import java.util.HashMap;

@WebService(endpointInterface = "com.poly.info.service.PersonService")


public class PersonServiceImpl implements PersonService {

    private static Map<Integer,Person> persons = new HashMap<Integer,Person>();

    @Override
    public boolean addPerson(Person p) {
        if(persons.get(p.getId()) != null) return false;
        persons.put(p.getId(), p);
        return true;
    }

    @Override
    public boolean deletePerson(int id) {
        if(persons.get(id) == null) return false;
        persons.remove(id);
        return true;
    }
```

```java
@Override
public Person getPerson(int id) {
    return persons.get(id);
}

@Override
public Person getPersonByName(String name) {
    for(int i = 1 ; i<=persons.size(); i++)
{
    if(persons.get(i).getName().equals(name))
        System.out.println(persons.get(i).toString());
        return persons.get(i);

}
     System.out.println("person doesn't exist");
     return null;

}

@Override
public Person[] getAllPersons() {
    Set<Integer> ids = persons.keySet();
    Person[] p = new Person[ids.size()];
    int i=0;
    for(Integer id : ids){
        p[i] = persons.get(id);
        i++;
    }
    return p;
}
```

La partie la plus importante est l'annotation `@WebService` où nous fournissons la valeur ***endpointInterface*** entant qu'interface pour notre service Web. Ainsi, **JAX-WS** connaît la classe à utiliser pour l'implémentation lorsque des méthodes de service Web sont appelées.

3) Notre logique métier de service Web est prête, publions-la à l'aide de la classe **JAX-WS Endpoint**, en créant une classe nommée :

`SOAPPublisher.java`

```java
import javax.xml.ws.Endpoint;

public class SOAPPublisher {

    public static void main(String[] args) {

        Endpoint.publish("http://localhost:8888/ws/person", new PersonServiceImpl());

        System.out.println("server running");
    }

}
```

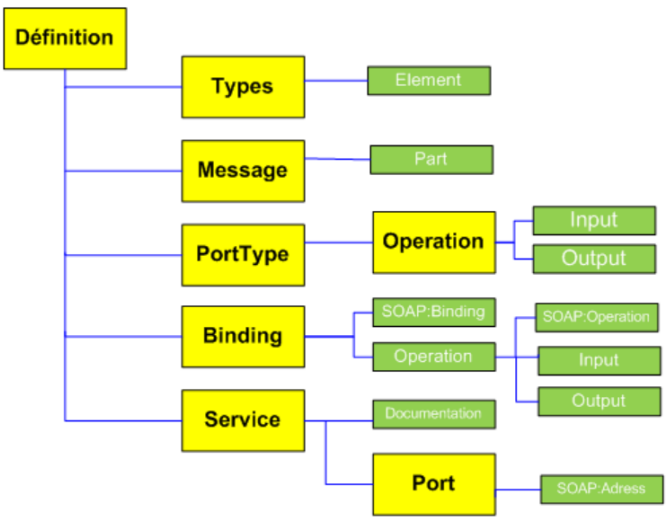
4) Il suffit d'exécuter le programme ci-dessus et votre service Web sera publié au point de terminaison (**endpoint**) donné dans le programme. Nous pouvons accéder à son document **WSDL** en ajoutant **«? wsdl »** à l'URL du endpoint, comme indiqué dans l'image ci-dessous.

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<!--
    Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e.
 -->
<!--
    Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e.
 -->
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
 xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
 xmlns:tns="http://service.info.poly.com/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://service.info.poly.com/"
 name="PersonServiceImplService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://service.info.poly.com/" schemaLocation="http://localhost:8888/ws/person?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="addPerson">
    <part name="arg0" type="tns:person"/>
  </message>
  <message name="addPersonResponse">
    <part name="return" type="xsd:boolean"/>
  </message>
  <message name="getPerson">
    <part name="arg0" type="xsd:int"/>
  </message>
  <message name="getPersonResponse">
    <part name="return" type="tns:person"/>
  </message>
  <message name="getPersonByName">
    <part name="arg0" type="xsd:string"/>
  </message>
  <message name="getPersonByNameResponse">
    <part name="return" type="tns:person"/>
  </message>
  <message name="getAllPersons"/>
  <message name="getAllPersonsResponse">
```


Activer Windows

Voici la structure d'un document WSDL :



Structure d'un document WSDL

5) On passe à écrire le programme client dans lequel nous appelons notre exemple de service Web JAX-WS. Puis, exécutons le programme client JAX-WS. Il y a trois classes importantes dans ce code :

a. La classe **URL (String spec)** qui permet de préciser l'adresse de notre service. En effet, la classe **URL** représente un Uniform Resource Locator, un pointeur vers une "ressource" sur le World Wide Web. Une ressource peut être quelque chose d'aussi simple qu'un fichier ou un répertoire, ou il peut s'agir d'une référence à un objet plus compliqué, comme une requête à une base de données.

b. La classe **QName (String namespaceURI, String localPart)** spécife l'URI de l'espace de noms et la partie locale, tel que défini dans le document wsdl.

c. La classe **Service** avec la méthode **create(arg1, arg2)** permet de créer notre service.

SOAPPublisherClient.java

```
3
4  import java.net.MalformedURLException;
12
13 public class SOAPPublisherClient {
14
15     public static void main(String[] args) throws MalformedURLException {
16
17         URL wsdlURL = new URL("http://localhost:8888/ws/person?wsdl");
18         //check above URL in browser, you should see WSDL file
19
20         //creating QName using targetNamespace and name
21         QName qname = new QName("http://service.info.poly.com/", "PersonServiceImplService");
22         Service service = Service.create(wsdlURL, qname);
23
24         //We need to pass interface and model beans to client
25         PersonService ps = service.getPort(PersonService.class);
26
27         Person p1 = new Person(); p1.setName("Mouna"); p1.setId(1); p1.setAge(30);
28         Person p2 = new Person(); p2.setName("Sarra"); p2.setId(2); p2.setAge(25);
29         Person p3 = new Person(); p3.setName("Yosr"); p3.setId(3); p3.setAge(22);
30
31         //add person
32         System.out.println("Add Person Status="+ps.addPerson(p1));
33         System.out.println("Add Person Status="+ps.addPerson(p2));
34         System.out.println("Add Person Status="+ps.addPerson(p3));
```